```
from .deterministic import deterministic_seed, deterministic_choice


# ----------------------------------------------------------
# CATEGORY DETECTION
# ----------------------------------------------------------
CATEGORY_KEYWORDS = {
    "dress": ["dress", "gown", "maxi", "midi", "mini"],
    "top": ["top", "blouse", "shirt", "tank", "tee"],
    "bottom": ["pants", "trousers", "jeans", "skirt", "shorts"],
    "outerwear": ["coat", "jacket", "blazer", "trench"],
    "activewear": ["leggings", "sports bra", "workout", "performance"],
    "swim": ["swim", "bikini", "one-piece"],
    "accessory": ["bag", "belt", "scarf", "hat", "gloves"],
    "footwear": ["shoe", "boot", "heel", "sneaker", "flat"],
}

def detect_category(text: str) -> str:
    text_lower = text.lower()
    for category, keywords in CATEGORY_KEYWORDS.items():
        if any(k in text_lower for k in keywords):
            return category
    return "general"



# ----------------------------------------------------------
# PRODUCT TYPE DETECTION
```

```
# ---------------------------------------------------------
def detect_product_type(text: str) -> str:
    text_lower = text.lower()
    for category, keywords in CATEGORY_KEYWORDS.items():
        for k in keywords:
            if k in text_lower:
                return k
    return "product"



# ---------------------------------------------------------
# PERSONA DETECTION
# ---------------------------------------------------------
PERSONA_KEYWORDS = {
    "runway": ["runway", "editorial", "couture", "avant-garde"],
    "minimalist": ["minimal", "clean", "simple", "streamlined"],
    "romantic": ["soft", "feminine", "flowy", "delicate"],
    "bold": ["bold", "statement", "vibrant", "graphic"],
    "classic": ["timeless", "heritage", "traditional", "refined"],
}

def detect_persona(text: str) -> str:
    text_lower = text.lower()
    for persona, keywords in PERSONA_KEYWORDS.items():
        if any(k in text_lower for k in keywords):
            return persona
```

```
    # deterministic fallback
    seed = deterministic_seed(text)
    return deterministic_choice(seed, list(PERSONA_KEYWORDS.keys()), salt="persona")


# ----------------------------------------------------------
# TREND DETECTION
# ----------------------------------------------------------
TREND_KEYWORDS = {
    "metallic": ["metallic", "chrome", "foil", "shimmer"],
    "sheer": ["sheer", "mesh", "transparent"],
    "tailored": ["tailored", "structured", "sharp"],
    "oversized": ["oversized", "relaxed", "slouchy"],
    "athleisure": ["athleisure", "sporty", "performance"],
}

def detect_trend(text: str) -> str:
    text_lower = text.lower()
    for trend, keywords in TREND_KEYWORDS.items():
        if any(k in text_lower for k in keywords):
            return trend
    seed = deterministic_seed(text)
    return deterministic_choice(seed, list(TREND_KEYWORDS.keys()), salt="trend")


# ----------------------------------------------------------
```

```
# SEASON DETECTION
# ----------------------------------------------------------
SEASON_KEYWORDS = {
    "spring": ["spring", "floral", "bloom"],
    "summer": ["summer", "sun", "heat", "vacation"],
    "fall": ["fall", "autumn", "harvest"],
    "winter": ["winter", "snow", "cold", "holiday"],
}

def detect_season(text: str) -> str:
    text_lower = text.lower()
    for season, keywords in SEASON_KEYWORDS.items():
        if any(k in text_lower for k in keywords):
            return season
    seed = deterministic_seed(text)
    return deterministic_choice(seed, list(SEASON_KEYWORDS.keys()), salt="season")



# ----------------------------------------------------------
# VIBE DETECTION
# ----------------------------------------------------------
VIBE_KEYWORDS = {
    "soft_luxe": ["soft", "luxe", "velvet", "silky"],
    "hyper_modern": ["modern", "futuristic", "sleek"],
    "romantic": ["romantic", "delicate", "feminine"],
    "bold_graphic": ["bold", "graphic", "statement"],
```

```
}

def detect_vibe(text: str) -> str:
    text_lower = text.lower()
    for vibe, keywords in VIBE_KEYWORDS.items():
        if any(k in text_lower for k in keywords):
            return vibe
    seed = deterministic_seed(text)
    return deterministic_choice(seed, list(VIBE_KEYWORDS.keys()), salt="vibe")



# ----------------------------------------------------------
# MATERIAL DETECTION
# ----------------------------------------------------------
MATERIAL_KEYWORDS = {
    "cotton": ["cotton", "jersey"],
    "silk": ["silk", "satin"],
    "denim": ["denim", "jean"],
    "leather": ["leather", "faux leather"],
    "knit": ["knit", "ribbed", "sweater"],
}

def detect_material(text: str) -> str:
    text_lower = text.lower()
    for material, keywords in MATERIAL_KEYWORDS.items():
        if any(k in text_lower for k in keywords):
```

```
            return material
    seed = deterministic_seed(text)
    return deterministic_choice(seed, list(MATERIAL_KEYWORDS.keys()), salt="material")


# ---------------------------------------------------------
# SILHOUETTE DETECTION
# ---------------------------------------------------------
SILHOUETTE_KEYWORDS = {
    "fitted": ["fitted", "bodycon", "slim"],
    "relaxed": ["relaxed", "oversized", "loose"],
    "structured": ["structured", "tailored"],
    "flowy": ["flowy", "draped"],
}

def detect_silhouette(text: str) -> str:
    text_lower = text.lower()
    for sil, keywords in SILHOUETTE_KEYWORDS.items():
        if any(k in text_lower for k in keywords):
            return sil
    seed = deterministic_seed(text)
    return deterministic_choice(seed, list(SILHOUETTE_KEYWORDS.keys()),
salt="silhouette")


# ---------------------------------------------------------
```

```
# DETAIL DETECTION
# ----------------------------------------------------------
DETAIL_KEYWORDS = {
    "ruffle": ["ruffle", "frill"],
    "pleated": ["pleat", "pleated"],
    "embellished": ["embellished", "beaded", "sequin"],
    "cutout": ["cutout", "cut-out"],
    "asymmetric": ["asymmetric", "diagonal"],
}

def detect_details(text: str) -> str:
    text_lower = text.lower()
    for detail, keywords in DETAIL_KEYWORDS.items():
        if any(k in text_lower for k in keywords):
            return detail
    seed = deterministic_seed(text)
    return deterministic_choice(seed, list(DETAIL_KEYWORDS.keys()), salt="detail")
from .brand_banks import (
    ADJECTIVES,
    SENSORY_VERBS,
    CONFIDENCE_PHRASES,
    EDITORIAL_FRAMES
)
From ..utils.deterministic import (
    Deterministic_seed,
    Deterministic_choice,
```

```
    Deterministic_multi
)
From .persona_router import detect_persona
From .detection import (
    Detect_category,
    Detect_product_type,
    Detect_trend,
    Detect_season,
    Detect_vibe,
    Detect_material,
    Detect_silhouette,
    Detect_details
)

# ----------------------------------------------------------
# HELPER: SELECT EDITORIAL FRAME
# ----------------------------------------------------------
Def select_editorial_frame(seed: int, salt: str = “”):
    Return deterministic_choice(seed, EDITORIAL_FRAMES, salt=salt)



# ----------------------------------------------------------
# HELPER: SELECT VOCAB
# ----------------------------------------------------------
Def select_adj(seed: int, salt: str = “”):
    Return deterministic_choice(seed, ADJECTIVES, salt=salt)
```

```
Def select_verb(seed: int, salt: str = “”):
   Return deterministic_choice(seed, SENSORY_VERBS, salt=salt)

Def select_confidence(seed: int, salt: str = “”):
   Return deterministic_choice(seed, CONFIDENCE_PHRASES, salt=salt)



# ----------------------------------------------------------
# SHORT EDITORIAL (1–2 sentences)
# ----------------------------------------------------------
Def generate_short_editorial(text: str) -> str:
   Seed = deterministic_seed(text)

   Adj = select_adj(seed, “short_adj”)
   Verb = select_verb(seed, “short_verb”)
   Confidence = select_confidence(seed, “short_conf”)

   Return (
      F”{verb.capitalize()} with {adj} intention. “
      F”{confidence.capitalize()}.”
   )



# ----------------------------------------------------------
# MEDIUM EDITORIAL (2–4 sentences)
```

```
# ----------------------------------------------------------
Def generate_medium_editorial(text: str) -> str:
   Seed = deterministic_seed(text)

   Adj = select_adj(seed, “med_adj”)
   Verb = select_verb(seed, “med_verb”)
   Confidence = select_confidence(seed, “med_conf”)

   Frame = select_editorial_frame(seed, “med_frame”)

   Category = detect_category(text)
   Persona = detect_persona(text)

   Editorial = frame.format(
      Adj=adj,
      Mood=persona,
      Occasion=category
   )

   Return (
      F”{editorial} “
      F”{verb.capitalize()} with {adj} clarity. “
      F”{confidence.capitalize()}.”
   )
```

```
# ----------------------------------------------------------
# LONG EDITORIAL (5–8 sentences)
# ----------------------------------------------------------
Def generate_long_editorial(text: str) -> str:
  Seed = deterministic_seed(text)

  Adj = select_adj(seed, “long_adj”)
  Verb = select_verb(seed, “long_verb”)
  Confidence = select_confidence(seed, “long_conf”)

  Frame1 = select_editorial_frame(seed, “long_frame1”)
  Frame2 = select_editorial_frame(seed, “long_frame2”)

  Category = detect_category(text)
  Persona = detect_persona(text)
  Trend = detect_trend(text)
  Season = detect_season(text)
  Vibe = detect_vibe(text)

  Part1 = frame1.format(
    Adj=adj,
    Mood=vibe,
    Occasion=category
  )

  Part2 = frame2.format(
```

```
        Adj=adj,
        Mood=persona,
        Occasion=season
    )

    Return (
        F”{part1} “
        F”{part2} “
        F”{verb.capitalize()} with {adj} depth, shaped by a {trend} sensibility. “
        F”{confidence.capitalize()}.”
    )


# ----------------------------------------------------------
# PERSONA-AWARE EDITORIAL
# ----------------------------------------------------------
Def generate_persona_editorial(text: str) -> str:
    Persona = detect_persona(text)
    Base = generate_long_editorial(text)

    Return f”{base} This piece embodies the {persona} spirit.”


# ----------------------------------------------------------
# CATEGORY-AWARE EDITORIAL
# ----------------------------------------------------------
```

```
Def generate_category_editorial(text: str) -> str:
    Category = detect_category(text)
    Base = generate_medium_editorial(text)

    Return f"{base} Designed to elevate your {category} wardrobe."



# ----------------------------------------------------------
# SEO-AWARE EDITORIAL
# ----------------------------------------------------------
Def generate_seo_editorial(text: str, seo_phrase: str) -> str:
    Base = generate_long_editorial(text)
    Return f"{base} Optimized for searches like: {seo_phrase}."
From .brand_banks import (
    ADJECTIVES,
    SENSORY_VERBS,
    CONFIDENCE_PHRASES
)
From ..utils.deterministic import (
    Deterministic_seed,
    Deterministic_choice,
    Deterministic_multi
)
From .persona_router import detect_persona
From .detection import (
    Detect_category,
```

```
    Detect_product_type,
    Detect_trend,
    Detect_season,
    Detect_vibe,
    Detect_material,
    Detect_silhouette,
    Detect_details
)

# ----------------------------------------------------------
# VOCAB HELPERS
# ----------------------------------------------------------
Def alt_adj(seed: int, salt: str = “”):
    Return deterministic_choice(seed, ADJECTIVES, salt=salt)

Def alt_verb(seed: int, salt: str = “”):
    Return deterministic_choice(seed, SENSORY_VERBS, salt=salt)

Def alt_conf(seed: int, salt: str = “”):
    Return deterministic_choice(seed, CONFIDENCE_PHRASES, salt=salt)



# ----------------------------------------------------------
# SHORT ALT TEXT (8–12 words)
# ----------------------------------------------------------
Def generate_short_alt_text(text: str) -> str:
```

```
    Seed = deterministic_seed(text)

    Adj = alt_adj(seed, “alt_short_adj”)
    Verb = alt_verb(seed, “alt_short_verb”)
    Category = detect_category(text)

    Return f”{adj} {category} piece, {verb} with intention.”



# ----------------------------------------------------------
# MEDIUM ALT TEXT (12–20 words)
# ----------------------------------------------------------
Def generate_medium_alt_text(text: str) -> str:
    Seed = deterministic_seed(text)

    Adj = alt_adj(seed, “alt_med_adj”)
    Verb = alt_verb(seed, “alt_med_verb”)
    Vibe = detect_vibe(text)
    Silhouette = detect_silhouette(text)
    Category = detect_category(text)

    Return (
        F”{adj} {category} in a {silhouette} silhouette, “
        F”{verb} with {vibe} energy.”
    )
```

```
# ----------------------------------------------------------
# LONG ALT TEXT (20–35 words)
# ----------------------------------------------------------
Def generate_long_alt_text(text: str) -> str:
   Seed = deterministic_seed(text)

   Adj = alt_adj(seed, “alt_long_adj”)
   Verb = alt_verb(seed, “alt_long_verb”)
   Vibe = detect_vibe(text)
   Material = detect_material(text)
   Detail = detect_details(text)
   Category = detect_category(text)

   Return (
      F”{adj} {category} crafted in {material} with {detail} detailing, “
      F”{verb} through a {vibe} aesthetic for elevated presence.”
   )



# ----------------------------------------------------------
# PERSONA-AWARE ALT TEXT
# ----------------------------------------------------------
Def generate_persona_alt_text(text: str) -> str:
   Persona = detect_persona(text)
   Base = generate_medium_alt_text(text)
```

```
    Return f"{base} Styled with a {persona} sensibility."



# ----------------------------------------------------------
# CATEGORY-AWARE ALT TEXT
# ----------------------------------------------------------
Def generate_category_alt_text(text: str) -> str:
    Category = detect_category(text)
    Base = generate_short_alt_text(text)

    Return f"{base} Ideal for {category} styling."



# ----------------------------------------------------------
# SEO-AWARE ALT TEXT
# ----------------------------------------------------------
Def generate_seo_alt_text(text: str, seo_phrase: str) -> str:
    Base = generate_long_alt_text(text)
    Return f"{base} Optimized for searches like: {seo_phrase}."
From ..utils.deterministic import (
    Deterministic_seed,
    Deterministic_choice,
    Deterministic_multi
)
From .detection import (
```

```
    Detect_category,
    Detect_product_type,
    Detect_trend,
    Detect_season,
    Detect_vibe,
    Detect_material,
    Detect_silhouette,
    Detect_details
)

# ----------------------------------------------------------
# SEO PRIMARY PHRASE
# ----------------------------------------------------------
Def generate_seo_primary(text: str) -> str:
    Seed = deterministic_seed(text)

    Category = detect_category(text)
    Product_type = detect_product_type(text)
    Vibe = detect_vibe(text)

    Options = [
        F”{vibe} {product_type}”,
        F”{category} {product_type}”,
        F”{vibe} {category} {product_type}”,
        F”{product_type} for {category} styling”
    ]
```

```
    Return deterministic_choice(seed, options, salt="seo_primary")


# ----------------------------------------------------------
# SEO SECONDARY PHRASE
# ----------------------------------------------------------
Def generate_seo_secondary(text: str) -> str:
    Seed = deterministic_seed(text)

    Trend = detect_trend(text)
    Season = detect_season(text)
    Material = detect_material(text)

    Options = [
        F"{season} {trend} {material}",
        F"{material} {trend} style",
        F"{season} {material} look",
        F"{trend} {material} outfit"
    ]

    Return deterministic_choice(seed, options, salt="seo_secondary")


# ----------------------------------------------------------
# SEO KEYWORD EXPANSION
```

```
# ----------------------------------------------------------
Def generate_seo_keywords(text: str) -> list:
    Seed = deterministic_seed(text)

    Category = detect_category(text)
    Product_type = detect_product_type(text)
    Trend = detect_trend(text)
    Season = detect_season(text)
    Vibe = detect_vibe(text)
    Material = detect_material(text)

    Base_keywords = [
        Category,
        Product_type,
        Trend,
        Season,
        Vibe,
        Material,
        F”{season} {product_type}”,
        F”{trend} {product_type}”,
        F”{material} {product_type}”,
        F”{vibe} {product_type}”,
    ]

    # deterministic multi-select
    Return deterministic_multi(seed, base_keywords, count=6, salt=”seo_keywords”)
```

```
# ----------------------------------------------------------
# SEO TAG GENERATOR
# ----------------------------------------------------------
Def generate_seo_tags(text: str) -> list:
    Keywords = generate_seo_keywords(text)
    Return [kw.replace(“ “, “_”).lower() for kw in keywords]



# ----------------------------------------------------------
# SEO COLLECTION GENERATOR
# ----------------------------------------------------------
Def generate_seo_collections(text: str) -> list:
    Seed = deterministic_seed(text)

    Category = detect_category(text)
    Trend = detect_trend(text)
    Season = detect_season(text)

    Options = [
        F”{season.capitalize()} {category.capitalize()} Edit”,
        F”{trend.capitalize()} Trends”,
        F”{category.capitalize()} Essentials”,
        F”{season.capitalize()} Statement Pieces”,
        F”{trend.capitalize()} Capsule Collection”
```

```
    ]

    Return deterministic_multi(seed, options, count=2, salt="seo_collections")
From ..utils.deterministic import (
    Deterministic_seed,
    Deterministic_choice,
    Deterministic_multi
)
From .detection import (
    Detect_category,
    Detect_product_type,
    Detect_trend,
    Detect_season,
    Detect_vibe,
    Detect_material,
    Detect_silhouette,
    Detect_details
)
From .seo_keywords import generate_seo_tags



# ----------------------------------------------------------
# CATEGORY TAGS
# ----------------------------------------------------------
Def generate_category_tags(text: str) -> list:
    Category = detect_category(text)
```

```
    Product_type = detect_product_type(text)

    Return [
        Category.lower(),
        Product_type.lower(),
        F”{category}_{product_type}”.lower()
    ]



# ----------------------------------------------------------
# PERSONA TAGS
# ----------------------------------------------------------
Def generate_persona_tags(text: str) -> list:
    Seed = deterministic_seed(text)
    Persona = deterministic_choice(seed, [
        “runway_edit”,
        “minimalist_edit”,
        “romantic_edit”,
        “bold_edit”,
        “classic_edit”
    ], salt=”persona_tags”)

    Return [persona]



# ----------------------------------------------------------
```

```
# TREND TAGS
# ----------------------------------------------------------
Def generate_trend_tags(text: str) -> list:
    Trend = detect_trend(text)
    Return [trend.lower(), f”trend_{trend}”.lower()]



# ----------------------------------------------------------
# SEASON TAGS
# ----------------------------------------------------------
Def generate_season_tags(text: str) -> list:
    Season = detect_season(text)
    Return [season.lower(), f”{season}_season”.lower()]



# ----------------------------------------------------------
# VIBE TAGS
# ----------------------------------------------------------
Def generate_vibe_tags(text: str) -> list:
    Vibe = detect_vibe(text)
    Return [vibe.lower(), f”vibe_{vibe}”.lower()]



# ----------------------------------------------------------
# MATERIAL TAGS
# ----------------------------------------------------------
```

```
Def generate_material_tags(text: str) -> list:
   Material = detect_material(text)
   Return [material.lower(), f”{material}_material”.lower()]


# ----------------------------------------------------------
# SILHOUETTE TAGS
# ----------------------------------------------------------
Def generate_silhouette_tags(text: str) -> list:
   Silhouette = detect_silhouette(text)
   Return [silhouette.lower(), f”{silhouette}_fit”.lower()]


# ----------------------------------------------------------
# DETAIL TAGS
# ----------------------------------------------------------
Def generate_detail_tags(text: str) -> list:
   Detail = detect_details(text)
   Return [detail.lower(), f”{detail}_detail”.lower()]


# ----------------------------------------------------------
# EDITORIAL TAGS
# ----------------------------------------------------------
Def generate_editorial_tags(text: str) -> list:
   Seed = deterministic_seed(text)
```

```
    Editorial_tags = [
        “luxury_edit”,
        “premium_edit”,
        “signature_edit”,
        “elevated_style”,
        “modern_glow”
    ]

    Return deterministic_multi(seed, editorial_tags, count=2, salt=”editorial_tags”)



# ----------------------------------------------------------
# ALT-TEXT TAGS
# ----------------------------------------------------------
Def generate_alt_text_tags(text: str) -> list:
    Seed = deterministic_seed(text)

    Alt_tags = [
        “visual_focus”,
        “image_ready”,
        “alt_text_optimized”,
        “accessibility_ready”,
        “visual_identity”
    ]
```

```
    Return deterministic_multi(seed, alt_tags, count=2, salt=”alt_tags”)



# ----------------------------------------------------------
# MASTER TAG AGGREGATOR
# ----------------------------------------------------------
Def generate_all_tags(text: str) -> list:
    Tags = []

    Tags += generate_category_tags(text)
    Tags += generate_persona_tags(text)
    Tags += generate_trend_tags(text)
    Tags += generate_season_tags(text)
    Tags += generate_vibe_tags(text)
    Tags += generate_material_tags(text)
    Tags += generate_silhouette_tags(text)
    Tags += generate_detail_tags(text)
    Tags += generate_editorial_tags(text)
    Tags += generate_alt_text_tags(text)
    Tags += generate_seo_tags(text)

    # Deduplicate while preserving order
    Seen = set()
    Final = []
    For t in tags:
        If t not in seen:
```

```
        Seen.add(t)
        Final.append(t)

    Return final
From ..brand_brain.brand_language import (
    Generate_short_editorial,
    Generate_medium_editorial,
    Generate_long_editorial
)
From ..brand_brain.alt_text import (
    Generate_short_alt_text,
    Generate_medium_alt_text,
    Generate_long_alt_text
)
From ..brand_brain.seo_keywords import (
    Generate_seo_primary,
    Generate_seo_secondary
)
From ..brand_brain.detection import (
    Detect_category,
    Detect_product_type,
    Detect_trend,
    Detect_season,
    Detect_vibe,
    Detect_material,
    Detect_silhouette,
```

```
    Detect_details
)
From ..brand_brain.persona_router import detect_persona



# ----------------------------------------------------------
# EDITORIAL METAFIELDS
# ----------------------------------------------------------
Def editorial_metafields(text: str) -> dict:
    Return {
        “editorial_short”: generate_short_editorial(text),
        “editorial_medium”: generate_medium_editorial(text),
        “editorial_long”: generate_long_editorial(text)
    }



# ----------------------------------------------------------
# ALT-TEXT METAFIELDS
# ----------------------------------------------------------
Def alt_text_metafields(text: str) -> dict:
    Return {
        “alt_short”: generate_short_alt_text(text),
        “alt_medium”: generate_medium_alt_text(text),
        “alt_long”: generate_long_alt_text(text)
    }
```

```
# ----------------------------------------------------------
# PERSONA METAFIELD
# ----------------------------------------------------------
Def persona_metafield(text: str) -> dict:
    Persona = detect_persona(text)
    Return {“persona”: persona}



# ----------------------------------------------------------
# CATEGORY METAFIELDS
# ----------------------------------------------------------
Def category_metafields(text: str) -> dict:
    Return {
        “category”: detect_category(text),
        “product_type”: detect_product_type(text)
    }



# ----------------------------------------------------------
# TREND METAFIELDS
# ----------------------------------------------------------
Def trend_metafields(text: str) -> dict:
    Return {“trend”: detect_trend(text)}
```

```
# ----------------------------------------------------------
# SEASON METAFIELDS
# ----------------------------------------------------------
Def season_metafields(text: str) -> dict:
    Return {“season”: detect_season(text)}


# ----------------------------------------------------------
# VIBE METAFIELDS
# ----------------------------------------------------------
Def vibe_metafields(text: str) -> dict:
    Return {“vibe”: detect_vibe(text)}


# ----------------------------------------------------------
# MATERIAL METAFIELDS
# ----------------------------------------------------------
Def material_metafields(text: str) -> dict:
    Return {“material”: detect_material(text)}


# ----------------------------------------------------------
# SILHOUETTE METAFIELDS
# ----------------------------------------------------------
Def silhouette_metafields(text: str) -> dict:
    Return {“silhouette”: detect_silhouette(text)}
```

```
# ----------------------------------------------------------
# DETAIL METAFIELDS
# ----------------------------------------------------------
Def detail_metafields(text: str) -> dict:
   Return {“detail”: detect_details(text)}



# ----------------------------------------------------------
# SEO METAFIELDS
# ----------------------------------------------------------
Def seo_metafields(text: str) -> dict:
   Return {
      “seo_primary”: generate_seo_primary(text),
      “seo_secondary”: generate_seo_secondary(text)
   }



# ----------------------------------------------------------
# MASTER METAFIELD BUNDLE
# ----------------------------------------------------------
Def generate_all_metafields(text: str) -> dict:
   Metafields = {}


   Metafields.update(editorial_metafields(text))
```

```
    Metafields.update(alt_text_metafields(text))
    Metafields.update(persona_metafield(text))
    Metafields.update(category_metafields(text))
    Metafields.update(trend_metafields(text))
    Metafields.update(season_metafields(text))
    Metafields.update(vibe_metafields(text))
    Metafields.update(material_metafields(text))
    Metafields.update(silhouette_metafields(text))
    Metafields.update(detail_metafields(text))
    Metafields.update(seo_metafields(text))

    Return metafields
From ..utils.deterministic import (
    Deterministic_seed,
    Deterministic_choice,
    Deterministic_multi
)
From .detection import (
    Detect_category,
    Detect_product_type,
    Detect_trend,
    Detect_season,
    Detect_vibe,
    Detect_material,
    Detect_silhouette,
    Detect_details
```

```
)
From .seo_keywords import generate_seo_collections



# ----------------------------------------------------------
# CATEGORY COLLECTIONS
# ----------------------------------------------------------
Def generate_category_collections(text: str) -> list:
    Category = detect_category(text)
    Product_type = detect_product_type(text)

    Return [
        F”{category.capitalize()} Essentials”,
        F”{product_type.capitalize()} Edit”,
        F”{category.capitalize()} Capsule”
    ]



# ----------------------------------------------------------
# PERSONA COLLECTIONS
# ----------------------------------------------------------
Def generate_persona_collections(text: str) -> list:
    Seed = deterministic_seed(text)

    Persona_collections = [
        “Runway Edit”,
```

```
        “Minimalist Capsule”,
        “Romantic Moodboard”,
        “Bold Statement Edit”,
        “Classic Essentials”
    ]

    Return deterministic_multi(seed, persona_collections, count=2,
salt=”persona_collections”)



# ----------------------------------------------------------
# TREND COLLECTIONS
# ----------------------------------------------------------
Def generate_trend_collections(text: str) -> list:
    Trend = detect_trend(text)

    Return [
        F”{trend.capitalize()} Trends”,
        F”{trend.capitalize()} Spotlight”
    ]



# ----------------------------------------------------------
# SEASON COLLECTIONS
# ----------------------------------------------------------
Def generate_season_collections(text: str) -> list:
```

```
    Season = detect_season(text)

    Return [
        F”{season.capitalize()} Edit”,
        F”{season.capitalize()} Essentials”
    ]



# ----------------------------------------------------------
# VIBE COLLECTIONS
# ----------------------------------------------------------
Def generate_vibe_collections(text: str) -> list:
    Vibe = detect_vibe(text)

    Return [
        F”{vibe.replace(‘_’, ‘ ‘).title()} Edit”,
        F”{vibe.replace(‘_’, ‘ ‘).title()} Capsule”
    ]



# ----------------------------------------------------------
# BENEFIT COLLECTIONS
# ----------------------------------------------------------
Def generate_benefit_collections(text: str) -> list:
    Seed = deterministic_seed(text)
```

```
    Benefit_options = [
        “Comfort First”,
        “Effortless Style”,
        “Everyday Luxury”,
        “Statement Pieces”,
        “Travel Essentials”
    ]

    Return deterministic_multi(seed, benefit_options, count=2, salt=”benefit_collections”)



# ----------------------------------------------------------
# ROUTINE COLLECTIONS
# ----------------------------------------------------------
Def generate_routine_collections(text: str) -> list:
    Seed = deterministic_seed(text)

    Routine_options = [
        “Workday Edit”,
        “Weekend Capsule”,
        “Evening Essentials”,
        “Vacation Edit”,
        “Lounge & Leisure”
    ]

    Return deterministic_multi(seed, routine_options, count=2, salt=”routine_collections”)
```

```
# ----------------------------------------------------------
# MASTER COLLECTION BUNDLE
# ----------------------------------------------------------
Def generate_all_collections(text: str) -> list:
    Collections = []

    Collections += generate_category_collections(text)
    Collections += generate_persona_collections(text)
    Collections += generate_trend_collections(text)
    Collections += generate_season_collections(text)
    Collections += generate_vibe_collections(text)
    Collections += generate_benefit_collections(text)
    Collections += generate_routine_collections(text)
    Collections += generate_seo_collections(text)

    # Deduplicate while preserving order
    Seen = set()
    Final = []
    For c in collections:
        If c not in seen:
            Seen.add©
            Final.append©

    Return final
```

Input

→ detection

→ vocab selection

→ editorial

→ alt text

→ tags

→ metafields

→ collections

→ final output dict

```
From ..brand_brain.brand_language import (
    Generate_short_editorial,
    Generate_medium_editorial,
    Generate_long_editorial,
    Generate_persona_editorial,
    Generate_category_editorial,
    Generate_seo_editorial
)
From ..brand_brain.alt_text import (
    Generate_short_alt_text,
    Generate_medium_alt_text,
    Generate_long_alt_text,
    Generate_persona_alt_text,
    Generate_category_alt_text,
    Generate_seo_alt_text
)
From ..brand_brain.tags import generate_all_tags
```

```
From ..brand_brain.collections import generate_all_collections
From ..metafields.metafield_engine import generate_all_metafields
From ..brand_brain.seo_keywords import (
    Generate_seo_primary,
    Generate_seo_secondary
)
From ..brand_brain.detection import (
    Detect_category,
    Detect_product_type,
    Detect_trend,
    Detect_season,
    Detect_vibe,
    Detect_material,
    Detect_silhouette,
    Detect_details
)
From ..brand_brain.persona_router import detect_persona



# ------------------------------------------------------------
# MASTER PRODUCT PROCESSOR
# ------------------------------------------------------------
Def process_product(text: str) -> dict:
    “””
    Full end-to-end MVQueen Omniluxe Engine pipeline.
    Takes a product text input and returns a complete
```

Structured output with editorial, alt text, tags,

Metafields, collections, SEO, and detection metadata.

“””

DETECTION LAYER

Category = detect_category(text)

Product_type = detect_product_type(text)

Persona = detect_persona(text)

Trend = detect_trend(text)

Season = detect_season(text)

Vibe = detect_vibe(text)

Material = detect_material(text)

Silhouette = detect_silhouette(text)

Detail = detect_details(text)

SEO LAYER

Seo_primary = generate_seo_primary(text)

Seo_secondary = generate_seo_secondary(text)

EDITORIAL LAYER

```
Editorial_short = generate_short_editorial(text)
Editorial_medium = generate_medium_editorial(text)
Editorial_long = generate_long_editorial(text)
Editorial_persona = generate_persona_editorial(text)
Editorial_category = generate_category_editorial(text)
Editorial_seo = generate_seo_editorial(text, seo_primary)

# ----------------------------
# ALT TEXT LAYER
# ----------------------------
Alt_short = generate_short_alt_text(text)
Alt_medium = generate_medium_alt_text(text)
Alt_long = generate_long_alt_text(text)
Alt_persona = generate_persona_alt_text(text)
Alt_category = generate_category_alt_text(text)
Alt_seo = generate_seo_alt_text(text, seo_primary)

# ----------------------------
# TAGS + COLLECTIONS
# ----------------------------
Tags = generate_all_tags(text)
Collections = generate_all_collections(text)

# ----------------------------
# METAFIELDS
# ----------------------------
```

```
Metafields = generate_all_metafields(text)

# ----------------------------
# FINAL OUTPUT
# ----------------------------
Return {
  “input_text”: text,

  # Detection
  “category”: category,
  “product_type”: product_type,
  “persona”: persona,
  “trend”: trend,
  “season”: season,
  “vibe”: vibe,
  “material”: material,
  “silhouette”: silhouette,
  “detail”: detail,

  # SEO
  “seo_primary”: seo_primary,
  “seo_secondary”: seo_secondary,

  # Editorial
  “editorial_short”: editorial_short,
  “editorial_medium”: editorial_medium,
```

```
        “editorial_long”: editorial_long,
        “editorial_persona”: editorial_persona,
        “editorial_category”: editorial_category,
        “editorial_seo”: editorial_seo,

        # Alt Text
        “alt_short”: alt_short,
        “alt_medium”: alt_medium,
        “alt_long”: alt_long,
        “alt_persona”: alt_persona,
        “alt_category”: alt_category,
        “alt_seo”: alt_seo,

        # Tags + Collections
        “tags”: tags,
        “collections”: collections,

        # Metafields
        “metafields”: metafields
    }
From catalog_processor.processor import process_product
Import json

# ----------------------------------------------------------
# SIMPLE RUNNER
# ----------------------------------------------------------
```

```
Def run_engine_on_text(text: str):
    “””
    Runs the full MVQueen Omniluxe Engine on a single text input.
    Returns the full structured output.
    “””
    Return process_product(text)



# ------------------------------------------------------------
# PRETTY PRINT (OPTIONAL)
# ------------------------------------------------------------
Def pretty_print_output(output: dict):
    “””
    Prints the engine output in a clean, readable format.
    “””
    Print(json.dumps(output, indent=4, ensure_ascii=False))



# ------------------------------------------------------------
# CLI ENTRY POINT
# ------------------------------------------------------------
If __name__ == “__main__”:
    Print(“\nMVQueen Omniluxe Engine — Ready.\n”)
    User_input = input(“Enter product text: “).strip()

    If not user_input:
```

```
    Print("No input provided. Exiting.")
  Else:
    Result = run_engine_on_text(user_input we)
    Print("\n--- ENGINE OUTPUT ---\n")
    Pretty_print_output(result)
    Print("\n--- END ---\n")
Import json
From typing import List


# ----------------------------------------------------------
# BATCH PROCESSOR
# ----------------------------------------------------------
Def process_batch(products: List[str]) -> List[dict]:
  """
  Runs the engine on a list of product text inputs.
  Returns a list of structured outputs.
  """
  Results = []
  For text in products:
    Result = process_product(text)
    Results.append(result)
  Return results



# ----------------------------------------------------------
# SAVE RESULTS TO JSON (OPTIONAL)
```

```
# ----------------------------------------------------------
Def save_results_to_json(results: List[dict], filename: str = “engine_output.json”):
  “””
  Saves batch results to a JSON file.
  “””
  With open(filename, “w”, encoding=”utf-8”) as f:
    Json.dump(results, f, indent=4, ensure_ascii=False)



# ----------------------------------------------------------
# QUICK TEST HARNESS
# ----------------------------------------------------------
Def test_engine():
  “””
  Runs a quick test on sample product descriptions.
  Modify the list below to test your own products.
  “””
  Sample_products = [
    “Black satin slip dress with lace trim”,
    “Oversized cotton tee with minimalist print”,
    “Structured leather blazer with sharp shoulders”,
    “Romantic floral chiffon maxi skirt”,
    “Metallic mesh top with futuristic vibe”
  ]

  Results = process_batch(sample_products)
```

```
    Print(“\n--- TEST RESULTS ---\n”)
    For r in results:
        Print(json.dumps(r, indent=4, ensure_ascii=False))
        Print(“\n--------------------\n”)

    # Optional: save to file
    # save_results_to_json(results)
Import csv
From .processor import process_product



# ----------------------------------------------------------
# FLATTEN METAFIELDS FOR SHOPIFY
# ----------------------------------------------------------
Def flatten_metafields(metafields: dict) -> dict:
    “””
    Shopify metafields must be flattened into individual columns.
    Example:
        Metafields[“editorial_short”] -> “metafield.editorial_short”
    “””
    Flat = {}
    For key, value in metafields.items():
        Flat[f”metafield.{key}”] = value
    Return flat
```

```
# ----------------------------------------------------------
# CONVERT ENGINE OUTPUT TO SHOPIFY CSV ROW
# ----------------------------------------------------------
Def convert_to_csv_row(engine_output: dict) -> dict:
    “””
    Converts the full engine output into a Shopify-compatible CSV row.
    “””
    Row = {}

    # Core Shopify fields
    Row[“Title”] = engine_output.get(“input_text”, “”)
    Row[“Body (HTML)”] = engine_output.get(“editorial_long”, “”)
    Row[“Tags”] = “, “.join(engine_output.get(“tags”, []))
    Row[“Collections”] = “, “.join(engine_output.get(“collections”, []))

    # SEO fields
    Row[“SEO Title”] = engine_output.get(“seo_primary”, “”)
    Row[“SEO Description”] = engine_output.get(“seo_secondary”, “”)

    # Alt text
    Row[“Image Alt Text”] = engine_output.get(“alt_long”, “”)

    # Detection metadata
    Row[“Category”] = engine_output.get(“category”, “”)
    Row[“Product Type”] = engine_output.get(“product_type”, “”)
```

```
    Row[“Persona”] = engine_output.get(“persona”, “”)
    Row[“Trend”] = engine_output.get(“trend”, “”)
    Row[“Season”] = engine_output.get(“season”, “”)
    Row[“Vibe”] = engine_output.get(“vibe”, “”)
    Row[“Material”] = engine_output.get(“material”, “”)
    Row[“Silhouette”] = engine_output.get(“silhouette”, “”)
    Row[“Detail”] = engine_output.get(“detail”, “”)

    # Metafields
    Metafields = engine_output.get(“metafields”, {})
    Row.update(flatten_metafields(metafields))

    Return row



# ----------------------------------------------------------
# PROCESS MULTIPLE PRODUCTS INTO CSV ROWS
# ----------------------------------------------------------
Def convert_batch_to_csv_rows(products: list) -> list:
    Rows = []
    For text in products:
        Engine_output = process_product(text)
        Row = convert_to_csv_row(engine_output)
        Rows.append(row)
    Return rows
```

```
# ----------------------------------------------------------
# SAVE CSV FILE
# ----------------------------------------------------------
Def save_csv(rows: list, filename: str = “mvqueen_output.csv”):
    If not rows:
        Print(“No rows to save.”)
        Return

    Fieldnames = list(rows[0].keys())

    With open(filename, “w”, newline=””, encoding=”utf-8”) as f:
        Writer = csv.DictWriter(f, fieldnames=fieldnames)
        Writer.writeheader()
        Writer.writerows(rows)

    Print(f”CSV saved as {filename}”)
```